报告生成填写

背景：

企业中会需要填写很多报告，这些报告需要人手动填写和完成。并且企业的报告往往趋向于同质化（越大的企业越要求格式规范）。

如果能有一个Agent 处理掉这件事，就能够节约大量人力资源。

目前把工业报告的生成填写任务拆成两个大部分

1、报告生成Agent

现有情况，大的报告单次生成存在诸多问题，但是小的识别其实还可以。

比如说 我把一个报告丢给模型，告诉它你需要识别这个报告的所有内容，然后给我一个报告的框架，效果非常差。

理想版本1：

用户提交一个报告样例，系统就能够完全识别框架信息，并给予用户待填信息表单，当用户提供这些信息后，能够生成框架信息格式的报告。

理想版本2：

用户提交一批同类型的报告样例，系统能够完全识别工业报告原框架信息，并给予用户待填信息表单，当用户提供这些信息后，能够生成框架信息格式的报告。

任务边界

1. 输入边界：Agent 接收什么？用户提供的报告样例和用户自己标注的报告信息，包括公共和非公共的数据标签，连续数据输入的标签等。
2. 处理边界：Agent 能做什么？能够提取1中信息，生成报告的代码。
3. 输出边界：交付物是什么？一个报告模版的python代码，待填的数据表单。

报告生成v1版本：

前端展示word内容，用户对报告的所有内容进行打标。将打标完成后的数据连同原报告一块给Agent。

用旗舰模型调用 Agent 识别这些信息生成一份待提交的数据表单和生成报告的代码。

报告生成v2版本

理想版本1工业版本（落地）

用户将报告模版提交给系统，由用户自己选择哪些属于框架，哪些是待填信息，完成后，批量完成

实现步骤：

1.

2、报告填写 Agent

- 框架 背景 问题 很重要 分几步 第一步 23 现有情况 没解决 任务边界 研究问题 工程上的细节部分可以不管

任务边界

1. 输入边界：Agent 接收什么？待填的数据表单，用户提供的数据内容（企业知识库）
2. 处理边界：Agent 能做什么？能够提取1中信息，将信息填入到报告表单中
3. 输出边界：交付物是什么？填写好的word。
4. 任务目标：创建一个合理的工作流以适应报告填写的需求。

已有工作related work

工作工作	已经做了什么	和我们差在哪里
Sufficient Context, 2025	明确定义 RAG context 是否足以回答，并用于 selective generation/abstention	它是通用 QA/RAG，不是字段填充；没有我们这种 FieldSpec、字段级 evidence requirement、多格式 autofill benchmark
UAEval4RAG, 2024	研究 RAG 中 unanswerable 问题和拒答	偏不可回答 QA，不是字段自动填充，也不强调字段 evidence/localization
RAGBench, 2024	做 explainable RAG 评测	主要是 RAG 评价，不是字段级可填/拒填任务
MMDocRAG / UniDoc-Bench / VisDoM, 2025	多模态文档 RAG、证据检索、文档 QA	很接近“多格式文档 RAG”，但仍主要是 QA，不是选择性字段填充
RealKIE, 2024	企业文档 key information extraction，非常接近字段抽取	偏结构化抽取/KIE，不是 RAG evidence sufficiency + selective abstention
READoc, 2025	文档结构化抽取 benchmark	偏 PDF 到 Markdown/结构化内容，不是字段填充决策和证据充分性判断

UAEval4RAG 2025 ACL

背景：目前的RAG评估框架存在严重的评估误差

- 只关注能回答的问题：现有的基准测试主要集中在那些可以从知识库中找到的答案的“可回答问题上”只评估答案的准确性和相关性
- 忽视拒答的能力：它们忽略了RAG系统必须具备的另一项关键能力，如何恰当地拒绝无法回答的请求。
- 如果系统无法识别并拒绝这类问题，就会导致幻觉（胡编乱造）或安全漏洞（执行有害指令），这在实际应用中是不可接受的。

难点：如何生成高质量的、定制化的“无法回答问题”数据集

- 通用基准不适用：以前针对大模型的“无法回答”基准测试通常是一些通用的、静态的问题。
- 缺乏针对性：这些通用问题无法战队特定的外部知识库定制，如果直接使用，模型拒绝回答可能仅仅因为检索不到上下文，而不是它真正理解了问题本身是“无法回答的”
- 多样性不足：需要覆盖多种类型的无法回答的情况，而不仅仅是简单的“知识库外”问题。

方案：提出了UAEval4RAG框架

UAEval4RAG这是一个全新的、端到端的评估框架

1. 定义了六种无法回答的类别
    - 信息不足（宠物被允许吗？没有地点）
    - 错误预设（佐治亚第一家迪士尼什么时候开，实际上没有）
    - 无意义（错误拼写、逻辑混乱或胡言乱语）

- 模态受限（超出系统支持的输入）
- 安全顾虑（涉及有害、非法或隐私）
- 库外问题（与知识库主题相关但库中无答案）

2. 自动化数据合成流水线
   a. 前五类用大模型，根据定义和示例自动改写与生成针对特定文档的陷阱问题。
   b. 针对库外问题：通过提取知识库的关键短语，爬取最新的相关新闻生成qa对，然后验证这些答案不在原始知识库中，从而生成高难度的库外测试集。
   c. 新的评估指标
      i. 拒答率：客观衡量模型拒绝回答的比例
      ii. 可接受率：主观衡量回应是否符合人类偏好。
      iii. 联合分数：综合考量“可回答问题的正确率和不可回答问题的拒答率。

发现

- 不存在一套固定的 RAG 配置（如某种嵌入模型+某种检索器）能在所有数据集上同时优化“回答正确”和“拒答能力”。

Sufficient Context 2025 ICLR

背景：

- RAG的普及与局限：为大预言模型提供额外的上下文 已经显著的提高了模型在新领域的事实性和可验证性，但仍然有很多问题
- 理想与现实的差距：理想情况下，上下文如果包含足够信息，模型应该给出正确答案，如果信息不足，模型拒绝回答，但现实往往表现不佳
- 现有研究的盲区：虽然已有大量研究关注相关性，但是缺乏对上下文是否足够回答问题“这一核心概念的直接定义和评估。之前的工作通常假设提供的上下文足以回答问题，或者仅关注模型在面对无关信息时的鲁棒性。

问题：

论文旨在解决RAG系统中一个未被明确定义的关键问题：

- 分类模糊，不清楚错误究竟是因为模型为能利用检索到的上下文，还是因为上下文本身就不足以回答问题
- 行为偏差：模型在面对足够上下文时经常产生幻觉，而在面对不足上下文时又不能恰当地拒绝回答
- 开放域问答的挑战：如何在不知道标准答案的情况下，判断给定的上下文是否足以推导出答案。

难点：

- 如何形式化定义足够上下文。这不仅仅涉及上下文是否包含答案，还涉及多条推理，消除歧义等情况

- 可扩展的评估：在没有标准答案的情况下，如何大规模、准确的自动标注数据，判断上下文是否足够
- 模型行为的矛盾
  - 足够上下文时：模型本应该答对，却常因为无法提取或组合信息而答错。
  - 不足上下文时：模型本应该拒绝，却因为过度自信而强行编造答案。
- 干预的平衡：如何在不牺牲模型在闭卷能力的前提下，引导模型在开卷的环境下更好的利用上下文并减少幻觉。

解决方案：

- **提出足够上下文的定义，如果上下文c包含回答问题Q所需要的所有必要信息，则称该上下文为足够**
- 自动评估器 autorater
  - 开发了一个基于LLM的自动评分器，仅根据“问题+上下文”判断上下文是否足够（无需标准答案）
  - 该评估器在人工标注数据达到了93%的准确率。

深度发现问题：

即使是 SOTA 模型（如 Gemini, GPT-4o），在上下文足够时仍有 14-16% 的错误率

当上下文不足时，模型更多是产生幻觉而非拒绝回答。

改进：

利用足够上下文信号作为额外的干预手段，结合模型的自信心评分来决定是否生成答案

- 使用一个较小的干预模型来判断上下文是否足够
- 训练一个简单的逻辑回归模型，结合自信心和上下文充分性两个信号来预测幻觉
- 效果：该方法在多个模型上回答准确率提高了2%-10%

The Art of Saying No: Contextual Noncompliance in Language Models NeurIPS 2024

核心：拒绝“不该做”的事情，和学会“该做”的事情一样重要

研究背景 (Background)

- 安全对齐的现状： 目前，针对大语言模型（LLM）的研究主要集中在“安全性”上，即防止模型生成有害内容（如暴力、仇恨言论、隐私泄露等）。主流的基准测试（Benchmarks）也大多只关注这一维度的“拒绝”能力。
- 更广泛的“不合规”需求： 作者指出，除了安全风险，现实中还有大量其他场景需要模型“拒绝”直接回答。例如，当用户提出的问题包含错误的前提、信息不完整、或者超出了模型的能力范围（如生成视频、回答未来事件）时，模型也应该表现出“非合规”行为。

- 信任与体验： 如果模型在这些非安全场景下盲目顺从（Compliance），会导致生成虚假信息、产生幻觉或提供错误建议，从而损害用户体验和对AI的信任。

核心问题 (Problem)

这篇论文旨在解决两个核心研究问题（RQs）：

1. 评估现状（RQ1）： 当前最先进的模型（如 GPT-4, Llama-3 等）在面对非安全类的“不合规”请求时，表现如何？
2. 改进方法（RQ2）： 如何在不损害模型通用能力（General Capabilities）和不引起“过度拒绝”（Over-refusal，即连该回答的问题也不回答了）的前提下，训练模型学会恰当地“拒绝”？

主要难点 (Challenges)

作者在研究中发现，要实现理想的“情境性不合规”，面临以下几个主要难点：

- 现有模型的“高顺从率”： 评估发现，即使是 GPT-4 这样的顶级模型，在面对“信息不完整”或“无法支持”的请求时，依然有高达 30% 的顺从率。它们倾向于假设用户意图并直接回答，而不是要求澄清或拒绝，这容易导致错误信息的生成。
- 开源模型的“拟人化”倾向： 许多开源模型（如 Llama-2, Mistral）在面对“拟人化请求”（如“你有感情吗？”）时，更容易表现出顺从行为（即承认自己有情感），这可能引发伦理和信任问题。
- 训练的平衡难题： 简单地通过监督微调（SFT）让模型学会拒绝，会导致两个副作用：
  - 灾难性遗忘： 模型会忘记原本具备的通用问答能力。
  - 过度拒绝（Over-refusal）： 模型变得过于敏感，连那些本该正常回答的良性问题（Contrast Set）也拒绝回答。

解决方案 (Solution)

为了解决上述问题，作者提出了一套完整的框架，包括一个新的分类体系、一个数据集和一套训练策略：

A. 构建基准：CoCoNoT

作者建立了一个名为 CoCoNoT 的基准测试集，包含 1000 个经过人工验证的“不合规”请求，涵盖了 5 大类场景：

1. Incomplete Requests (信息不完整): 如歧义问题、错误前提。
2. Unsupported Requests (无法支持): 如模态限制（无法生成图片）、长度限制。
3. Indeterminate Requests (无法确定): 如主观问题、未知知识。
4. Humanizing Requests (拟人化请求): 如询问模型的感受。
5. Safety Concerns (安全顾虑): 传统的有害请求。

B. 训练策略：LoRA + DPO

作者探索了多种训练方法，最终发现以下组合效果最佳：

1. 使用 LoRA 进行微调：
    - 不是对整个模型进行全参数微调，而是使用 低秩适配器 (LoRA)。
    - 效果： 这种方法在显著提高“不合规”能力的同时，最大程度地保留了模型的通用能力（避免了灾难性遗忘）。
2. 引入对比集与 DPO：
    - 为了防止“过度拒绝”，作者构建了对比集 (Contrast Set)，即那些看起来像坏问题但其实是好问题的数据。
    - 使用 DPO (直接偏好优化) 在这些对比集上进行训练。
    - 效果： 这能教会模型区分“真正的坏请求”和“表面像坏请求的良请求”，从而减少误拒。

核心贡献总结表

表格

维度	详细内容
核心洞察	拒绝不仅限于安全，还包括处理错误信息、能力边界和伦理问题。
主要发现	GPT-4 等模型在非安全类拒绝任务上仍有严重缺陷（最高30%误答）。
关键工具	CoCoNoT 数据集（含 Taxonomy 和 Evaluation Suite）。
最佳实践	LoRA 微调 + DPO (基于对比偏好数据)。
最终效果	在保持模型通用能力的同时，显著降低了幻觉和盲目顺从的风险。

简单来说，这篇论文教会了 AI 不仅要“有问必答”，更要学会在不该答的时候“说不”，并且找到了一种高效且不破坏模型原有能力的训练方法。

现有工作研究了 RAG context sufficiency、unanswerability、multimodal document QA 和 key information extraction，但它们通常将任务表述为 QA 或结构化抽取。本文提出 Evidence-Sufficient Selective Field Autofill，将问题重构为字段级自动填充决策：系统不仅要预测字段值，还必须判断当前检索证据是否足以支持自动填充，并在证据不足、缺失定位、操作不可计算或冲突时选择拒填/转人工。

我们的创新点应该是这几个组合：

1. 任务定义创新
   不是普通 QA，也不是普通 KIE，而是“字段级选择性填充”。
2. 数据集构建创新
   把 ContractNLI、CUAD、WikiTQ、TabFact、SpreadsheetBench、SROIE、FUNSD 统一转成同一种字段填充 schema。
3. 证据充分性标签体系
   fill_allowed、insufficient_evidence、unsupported_operation、missing_localization、needs_review 等，是面向字段填充场景的。
4. 真实 RAG 下的四象限实验
   尤其是：
   - 证据不足但答案正确
   - 证据充分但答案错误
   - 证据不足且答案错误
   - 证据充分且答案正确
5. 字段级 workflow
   不是只改 embedding 或 retriever，而是设计 Retrieval Agent、Evidence Verifier、Field Filler、Abstention/Review Decision 的流程。

字段自动填充场景下，系统何时应该填，何时不应该填。

科研版本实现步骤：

1. 建立知识库（材料）、问题和金标准数据集。 证据充分性（百分比）
   a. 目前没有跨文件的问题。
   b. 是否证据不充分也能回答出问题。
   c. 置信度指标。
2. 动机实验
   a. 四种结果
      i. RAG 检索不充分（低于某一置信度） 答案正确。（信息不全 乱填）
      ii. RAG 检索充分　答案错误。（信息全但 模型回答有误）
      iii. RAG 检索不充分 答案错误。（信息不全 模型错误）
      iv. RAG 检索充分　答案正确。（理想效果）
3. 统一选择 Embedding 和 RAG 检索方式？不在这两点上创新。
   a. 丰富实验内容，考虑多种检索方式。
4. 设计一套工作流，来实现这个任务。 预估将准确率从 20% 提升到 60%。

a. 构造 mas 和 subagent，分工减少模型的记忆导致的幻觉。

b. 据初始版本，根据问题检索答案。预期效果：大量错误的情况。

5. 构造经验总结。预估将准确率从60% 提升到 80%。
   这个是重做的理由。让模型能够在失败中经验重新组织关键词检索。
6. 消融实验，查看去掉某模块是否会降低成功率。
7. 鲁棒实验，单一的 embedding + rag 组合检索对比，应用我们的工作流后检索。
8. 对比实验，市面上的模型，不套工作流准确率和套工作流准确率。

工业版本实现步骤：

3、系统级实现